Maintenance and Support Plan

P2P Car Rental System

Course	MSE800 Professional Software Engineering
Assignment	Assignment 1 - Task 3
Version	1.0.0
Date	May 2026

1. Introduction

This document explains how the P2P Car Rental System will be maintained after the first release. Software needs regular work to fix bugs, add features, and stay compatible with new technology. Maintenance is a very important part of a project. Research shows that maintenance can take 60% to 90% of the total software cost (Erlikh, 2000; Glass, 2002). For this reason, we need a clear plan.

This plan has three main parts:

- Software maintenance (how we handle bugs and updates).
- Versioning (how we name each release).
- Backward compatibility (how we avoid breaking old features).

2. Software Maintenance

2.1 Types of Maintenance

The ISO/IEC 14764 standard defines four types of software maintenance (ISO/IEC, 2006). Our project will follow these types.

Type	Description	Example
Corrective	Fix bugs and errors found after release.	Fix a booking date overlap bug.
Adaptive	Update the software for new environments.	Support new Python versions.
Perfective	Improve features or performance.	Make the car search faster.
Preventive	Fix problems before they happen.	Refactor old code and add tests.

2.2 How We Handle Issues

When a user finds a bug or asks for a new feature, we follow these steps:

Step	What we do
1	User reports the issue (email or GitHub).
2	Team checks the issue and gives it a priority.
3	We create a ticket in the issue tracker.
4	A developer is assigned to the ticket.
5	The developer creates a branch and writes a fix.
6	Another developer reviews the code.
7	We run all tests to check that nothing is broken.
8	The fix is merged into the main branch.

Step	What we do
9	A new release is published with the fix.

2.3 Response Times

Each issue gets a priority. The table below shows the response time for each priority.

Priority	Examples	Response Time	Fix Time
Critical	System is down, data loss.	1 hour	4 hours
High	Cannot make a booking.	4 hours	24 hours
Medium	Small bug with a workaround.	1 day	1 week
Low	Small visual issue, typo.	3 days	Next release

2.4 Code Quality Tools

To keep the code clean, we use these tools:

- Git for version control. Every change is saved.
- GitHub Actions for automatic testing on every commit.
- pytest to run unit and integration tests.
- Black to format the code in a consistent style.
- Flake8 to find common Python mistakes.

3. Versioning

3.1 Semantic Versioning

We use Semantic Versioning 2.0.0 for our releases (Preston-Werner, 2013). Each version has three numbers in the format MAJOR.MINOR.PATCH.

Number	When to increase	Example
MAJOR	When we break backward compatibility.	1.0.0 to 2.0.0
MINOR	When we add new features but keep old ones working.	1.2.0 to 1.3.0
PATCH	When we fix bugs without changing features.	1.2.3 to 1.2.4

3.2 Release Schedule

We plan to release new versions on this schedule:

Type	Frequency	Content
Patch (1.0.x)	Every week	Bug fixes only.
Minor (1.x.0)	Every 4 to 6 weeks	New features, no breaking changes.
Major (x.0.0)	Every 12 to 18 months	Big changes, may break old code.

3.3 Git Tags and Changelog

Every release is tagged in Git with the version number. We also keep a CHANGELOG.md file. The format follows the "Keep a Changelog" standard (Oliver, 2017). Each release has four sections: Added, Changed, Fixed, and Removed.

Example commands for tagging a release:

```
git tag -a v1.0.0 -m "Initial release"
git push origin v1.0.0
```

4. Backward Compatibility

Backward compatibility means that new versions can still work with data, files, and code from older versions. This is important because users do not want their work to break when they update the software.

4.1 Three Pillars

Pillar	What it means	Example
Data compatibility	New code can read old database files.	A database from v1.0 still works in v1.5 after migration.
API compatibility	Old function names and parameters still work.	register(name, email) still works in v2.0.
User experience	The menu and workflow stay similar.	Old users do not need to learn everything again.

4.2 Database Migrations

When we change the database schema, we write a migration script. The script updates the old database to the new format without losing data.

For example, in version 1.2 we added a discount_code column to the bookings table. The migration script adds the column and sets the default value to NULL for old rows.

4.3 Deprecation Lifecycle

Sometimes we need to remove old features. We do not remove them right away. Instead, we follow a three-stage process:

Stage	What happens	How long
1. Announce	We tell users this feature will be removed soon.	6 months before removal.
2. Coexist	Old and new features both work. The old one shows a warning.	3 to 6 months.
3. Remove	The old feature is removed from the code.	In the next MAJOR version.

4.4 Testing for Compatibility

Before each release, we run special tests to check backward compatibility:

- Load a database from an older version and check that all data still works.
- Run the old example scripts to check that they still pass.
- Test the upgrade path from one version to the next.

5. Future Roadmap

The table below shows planned features for future versions. Dates are estimates and may change.

Version	Date	Main features
1.1.0	Q3 2026	Email notifications (SMTP).
1.2.0	Q4 2026	Loyalty points and tier discounts.
1.3.0	Q1 2027	Multi-language support.
2.0.0	Q2 2027	PostgreSQL backend and REST API.
2.1.0	Q3 2027	Mobile app (React Native).
2.2.0	Q4 2027	Payment gateway (Stripe).
3.0.0	Q2 2028	Smart matching with AI.

6. Conclusion

This plan covers the main parts of software maintenance for the P2P Car Rental System. The plan uses standard methods from the software industry. We will follow ISO/IEC 14764 for maintenance types, Semantic Versioning 2.0.0 for releases, and a clear deprecation process for old features. With this plan, the system can be updated and improved in a safe way.

References

Erlikh, L. (2000). Leveraging legacy system dollars for e-business. *IT Professional*, 2(3), 17-23.

Glass, R. L. (2002). *Facts and Fallacies of Software Engineering*. Boston: Addison-Wesley.

ISO/IEC (2006). *ISO/IEC 14764:2006 Software Engineering - Software Life Cycle Processes - Maintenance*. Geneva: International Organization for Standardization.

Oliver, O. (2017). *Keep a Changelog 1.0.0*. Retrieved from https://keepachangelog.com/

Preston-Werner, T. (2013). *Semantic Versioning 2.0.0*. Retrieved from https://semver.org/